Viste `dw_*` — Sorgenti e Query (riferimento sintetico)

Per ogni vista: **cosa rappresenta**, **tabelle sorgenti** e **query SQL** usata per generarla.
Schema di destinazione: `public`. Sorgenti: `dwh` = questionari (EAV), `ca` = Conti Annuali.
Collegamento ente comune alle viste dei Conti Annuali: `ca.<fatto>.ISTITUZIONE` = `ca.lk_istituzioni.istituzione_id`; `ca.lk_istituzioni.CODI_FISCALE` = `dwh.lk_questionari_enti.codice_fiscale_ente` -> `id` (= `istituzione`).

Indice: vista -> tabelle sorgenti

Vista	Tabelle sorgenti
`dw_ente`	`dwh.lk_questionari_enti`, `dwh.lk_comuni`
`v_kpi_ente_wide`	`dwh.ft_questionari_globale`, `dwh.lk_questionari_enti`
`dw_kpi_rilevazione`	`public.v_kpi_ente_wide`, `public.dw_ente`
`dw_verifica_indicatori`	`public.dw_ente`, `public.dw_kpi_rilevazione`
`dw_occupazione`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_occupazione`
`dw_assunti`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_assunzioni`
`dw_cessati`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_cessazioni`
`dw_eta`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_eta`
`dw_formazione`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_formazione`
`dw_modalita_lavoro`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_modalita_lavoro_flessibile`
`dw_passaggi_qualifica`	`ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_passaggi_qualifica`
`dw_causali`	`ca.vw_lk_causali_assunzione`, `ca.vw_lk_causali_cessazione`
`dw_comparto_contratto`	`ca.lk_mappa_comparti_contratti`
`dw_fascia_eta`	*(derivata, nessuna tabella)*
`dw_qualifiche`	`ca.lk_comparti_categorie_contratti`

`public.dw_ente`

Cosa rappresenta: Anagrafica enti monitorati

Tabelle sorgenti: `dwh.lk_questionari_enti`, `dwh.lk_comuni`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_ente CASCADE;
CREATE MATERIALIZED VIEW public.dw_ente AS
SELECT
   e.id::int                        AS id_ente,
   e.codice_fiscale_ente            AS cfiscale,
   e.codice_ipa                     AS codice_ipa,
   e.denominazione_ente             AS denominazione,
   e.tipologia                      AS tipo_istituzione,
   e.codice_categoria               AS cod_tipo,
   NULL::text                       AS comparto,               -- TODO: derivare (progetto/tipologia)
   NULL::text                       AS cod_comparto,           -- TODO
   NULL::text                       AS contratto,              -- TODO
   NULL::text                       AS cod_contratto,          -- TODO
   c."Regione"                      AS regione,
   e.tipologia                      AS classe_amministrazione, -- TODO: confermare
   e.classe_di_popolazione          AS categoria_cruscotto,    -- TODO: confermare mapping
   NULL::text                       AS profilo_prestazionale,  -- TODO
   NULL::int                        AS organico_2023,          -- TODO: da ft_questionari_globale (elemento
   1                                AS stato
FROM dwh.lk_questionari_enti e
LEFT JOIN dwh.lk_comuni c
      ON c."IstatID" = e.codice_comune_istat;
CREATE UNIQUE INDEX IF NOT EXISTS dw_ente_id_ente_idx ON public.dw_ente (id_ente);
```

`public.v_kpi_ente_wide`

Cosa rappresenta: Pivot base questionari EAV -> colonne (helper interno)

Tabelle sorgenti: `dwh.ft_questionari_globale`, `dwh.lk_questionari_enti`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.v_kpi_ente_wide CASCADE;
CREATE MATERIALIZED VIEW public.v_kpi_ente_wide AS
SELECT
 e.id::int            AS id_ente,
 f.cf_ente            AS cfiscale,
 f.rilevazione        AS rilevazione,
 f.progetto           AS progetto,
 MAX(CASE WHEN f.raw_element = '1.1' THEN f.valore_num END) AS "q_1_1",
 MAX(CASE WHEN f.raw_element = '1.2' THEN f.valore_num END) AS "q_1_2",
 MAX(CASE WHEN f.raw_element = '1.2.a' THEN f.valore_num END) AS "q_1_2_a",
 MAX(CASE WHEN f.raw_element = '1.2.b' THEN f.valore_num END) AS "q_1_2_b",
 MAX(CASE WHEN f.raw_element = '1.3' THEN f.valore_num END) AS "q_1_3",
 MAX(CASE WHEN f.raw_element = '1.4' THEN f.valore_num END) AS "q_1_4",
 MAX(CASE WHEN f.raw_element = '1.5' THEN f.valore_num END) AS "q_1_5",
 MAX(CASE WHEN f.raw_element = '1.6' THEN f.valore_num END) AS "q_1_6",
 MAX(CASE WHEN f.raw_element = '2.1' THEN f.valore_num END) AS "q_2_1",
 MAX(CASE WHEN f.raw_element = '2.2' THEN f.valore_num END) AS "q_2_2",
 MAX(CASE WHEN f.raw_element = '2.3' THEN f.valore_num END) AS "q_2_3",
 MAX(CASE WHEN f.raw_element = '2.4' THEN f.valore_num END) AS "q_2_4",
 MAX(CASE WHEN f.raw_element = '2.5' THEN f.valore_num END) AS "q_2_5",
 MAX(CASE WHEN f.raw_element = '2.6' THEN f.valore_num END) AS "q_2_6",
 MAX(CASE WHEN f.raw_element = '3.1' THEN f.valore_num END) AS "q_3_1",
 MAX(CASE WHEN f.raw_element = '3.2' THEN f.valore_num END) AS "q_3_2",
 MAX(CASE WHEN f.raw_element = '3.3' THEN f.valore_num END) AS "q_3_3",
 MAX(CASE WHEN f.raw_element = '3.4' THEN f.valore_num END) AS "q_3_4",
 MAX(CASE WHEN f.raw_element = '3.5' THEN f.valore_num END) AS "q_3_5",
 MAX(CASE WHEN f.raw_element = '3.6' THEN f.valore_num END) AS "q_3_6",
 MAX(CASE WHEN f.raw_element = '4.1' THEN f.valore_num END) AS "q_4_1",
 MAX(CASE WHEN f.raw_element = '4.2' THEN f.valore_num END) AS "q_4_2",
 MAX(CASE WHEN f.raw_element = '4.3' THEN f.valore_num END) AS "q_4_3",
 MAX(CASE WHEN f.raw_element = '5.1' THEN f.valore_num END) AS "q_5_1",
 MAX(CASE WHEN f.raw_element = '5.2' THEN f.valore_num END) AS "q_5_2",
```

```
  MAX(CASE WHEN f.raw_element = '5.3' THEN f.valore_num END) AS "q_5_3",
  MAX(CASE WHEN f.raw_element = '6.1' THEN f.valore_num END) AS "q_6_1",
  MAX(CASE WHEN f.raw_element = '6.10' THEN f.valore_num END) AS "q_6_10",
  MAX(CASE WHEN f.raw_element = '6.10.a' THEN f.valore_num END) AS "q_6_10_a",
  MAX(CASE WHEN f.raw_element = '6.10.b' THEN f.valore_num END) AS "q_6_10_b",
  MAX(CASE WHEN f.raw_element = '6.11' THEN f.valore_num END) AS "q_6_11",
  MAX(CASE WHEN f.raw_element = '6.11.a' THEN f.valore_num END) AS "q_6_11_a",
  MAX(CASE WHEN f.raw_element = '6.11.b' THEN f.valore_num END) AS "q_6_11_b",
  MAX(CASE WHEN f.raw_element = '6.12.a' THEN f.valore_num END) AS "q_6_12_a",
  MAX(CASE WHEN f.raw_element = '6.12.b' THEN f.valore_num END) AS "q_6_12_b",
  MAX(CASE WHEN f.raw_element = '6.13' THEN f.valore_num END) AS "q_6_13",
  MAX(CASE WHEN f.raw_element = '6.13.a' THEN f.valore_num END) AS "q_6_13_a",
  MAX(CASE WHEN f.raw_element = '6.13.b' THEN f.valore_num END) AS "q_6_13_b",
  MAX(CASE WHEN f.raw_element = '6.14.a' THEN f.valore_num END) AS "q_6_14_a",
  MAX(CASE WHEN f.raw_element = '6.14.b' THEN f.valore_num END) AS "q_6_14_b",
  MAX(CASE WHEN f.raw_element = '6.15' THEN f.valore_num END) AS "q_6_15",
  MAX(CASE WHEN f.raw_element = '6.16' THEN f.valore_num END) AS "q_6_16",
  MAX(CASE WHEN f.raw_element = '6.16.a' THEN f.valore_num END) AS "q_6_16_a",
  MAX(CASE WHEN f.raw_element = '6.16.b' THEN f.valore_num END) AS "q_6_16_b",
  MAX(CASE WHEN f.raw_element = '6.17' THEN f.valore_num END) AS "q_6_17",
  MAX(CASE WHEN f.raw_element = '6.17.a' THEN f.valore_num END) AS "q_6_17_a",
  MAX(CASE WHEN f.raw_element = '6.17.b' THEN f.valore_num END) AS "q_6_17_b",
  MAX(CASE WHEN f.raw_element = '6.17.c' THEN f.valore_num END) AS "q_6_17_c",
  MAX(CASE WHEN f.raw_element = '6.17.d' THEN f.valore_num END) AS "q_6_17_d",
  MAX(CASE WHEN f.raw_element = '6.18' THEN f.valore_num END) AS "q_6_18",
  MAX(CASE WHEN f.raw_element = '6.18.a' THEN f.valore_num END) AS "q_6_18_a",
  MAX(CASE WHEN f.raw_element = '6.18.b' THEN f.valore_num END) AS "q_6_18_b",
  MAX(CASE WHEN f.raw_element = '6.18.c' THEN f.valore_num END) AS "q_6_18_c",
  MAX(CASE WHEN f.raw_element = '6.18.d' THEN f.valore_num END) AS "q_6_18_d",
  MAX(CASE WHEN f.raw_element = '6.19.a' THEN f.valore_num END) AS "q_6_19_a",
  MAX(CASE WHEN f.raw_element = '6.19.b' THEN f.valore_num END) AS "q_6_19_b",
  MAX(CASE WHEN f.raw_element = '6.19.c' THEN f.valore_num END) AS "q_6_19_c",
  MAX(CASE WHEN f.raw_element = '6.19.d' THEN f.valore_num END) AS "q_6_19_d",
  MAX(CASE WHEN f.raw_element = '6.2' THEN f.valore_num END) AS "q_6_2",
  MAX(CASE WHEN f.raw_element = '6.20.a' THEN f.valore_num END) AS "q_6_20_a",
  MAX(CASE WHEN f.raw_element = '6.20.b' THEN f.valore_num END) AS "q_6_20_b",
  MAX(CASE WHEN f.raw_element = '6.20.c' THEN f.valore_num END) AS "q_6_20_c",
  MAX(CASE WHEN f.raw_element = '6.20.d' THEN f.valore_num END) AS "q_6_20_d",
  MAX(CASE WHEN f.raw_element = '6.3.a' THEN f.valore_num END) AS "q_6_3_a",
  MAX(CASE WHEN f.raw_element = '6.3.b' THEN f.valore_num END) AS "q_6_3_b",
  MAX(CASE WHEN f.raw_element = '6.4.a' THEN f.valore_num END) AS "q_6_4_a",
  MAX(CASE WHEN f.raw_element = '6.4.b' THEN f.valore_num END) AS "q_6_4_b",
  MAX(CASE WHEN f.raw_element = '6.4.c' THEN f.valore_num END) AS "q_6_4_c",
  MAX(CASE WHEN f.raw_element = '6.4.d' THEN f.valore_num END) AS "q_6_4_d",
  MAX(CASE WHEN f.raw_element = '6.5' THEN f.valore_num END) AS "q_6_5",
  MAX(CASE WHEN f.raw_element = '6.5.a' THEN f.valore_num END) AS "q_6_5_a",
  MAX(CASE WHEN f.raw_element = '6.5.b' THEN f.valore_num END) AS "q_6_5_b",
  MAX(CASE WHEN f.raw_element = '6.5.c' THEN f.valore_num END) AS "q_6_5_c",
  MAX(CASE WHEN f.raw_element = '6.5.d' THEN f.valore_num END) AS "q_6_5_d",
  MAX(CASE WHEN f.raw_element = '6.6' THEN f.valore_num END) AS "q_6_6",
  MAX(CASE WHEN f.raw_element = '6.7' THEN f.valore_num END) AS "q_6_7",
  MAX(CASE WHEN f.raw_element = '6.8' THEN f.valore_num END) AS "q_6_8",
  MAX(CASE WHEN f.raw_element = '6.9' THEN f.valore_num END) AS "q_6_9"
FROM dwh.ft_questionari_globale f
LEFT JOIN dwh.lk_questionari_enti e ON e.codice_fiscale_ente = f.cf_ente
GROUP BY e.id, f.cf_ente, f.rilevazione, f.progetto;
```

`public.dw_kpi_rilevazione`

Cosa rappresenta: KPI questionari per ente (~70 colonne q*)

Tabelle sorgenti: `public.v_kpi_ente_wide`, `public.dw_ente`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_kpi_rilevazione CASCADE;
CREATE MATERIALIZED VIEW public.dw_kpi_rilevazione AS
WITH w AS (
   SELECT *
   FROM public.v_kpi_ente_wide
   WHERE progetto = 'progetto_gru'
     AND rilevazione = '2025-12'
)
SELECT
   w.id_ente                                   AS id,
   w.id_ente                                   AS id_ente,
   w.cfiscale                                  AS cfiscale,
   de.denominazione                            AS denominazione,
   de.regione                                  AS regione,
   de.tipo_istituzione                         AS tipologia_amm,
   de.categoria_cruscotto                      AS dimensione_amm,
   NULL::text                                  AS segmento,        -- TODO: definire segmentazione
   w.rilevazione                               AS semestre,
   NULL::text                                  AS status,
   CASE WHEN w."q_1_1" = 1 THEN 'Formalmente' ELSE 'No' END            AS q1_1_adozione_modello,     -- raw
   CASE WHEN w."q_1_3" = 1 THEN 'Sì' ELSE 'No' END                     AS q1_2_library_processi,     -- raw
   CASE WHEN w."q_1_4" = 1 THEN 'Sì' ELSE 'No' END                     AS q1_3_dizionario_competenze,-- raw
   w."q_1_5"::text                                                      AS q1_5_n_profili_definiti,   -- raw
   w."q_1_6"::text                                                      AS q1_6_n_profili_competenze, -- raw
   w."q_1_5"::text                                                      AS q1_profili_totali,         -- raw
   w."q_2_2"::text                                                      AS q2_1_assunti_under35,      -- raw
   w."q_2_6"::text                                                      AS q2_1_assunzioni_turnover,  -- raw
   w."q_2_1"::text                                                      AS q2_2_assunti_ti,           -- raw
   w."q_2_4"::text                                                      AS q2_2_eq_ep_assunti,        -- raw
   w."q_2_3"::text                                                      AS q2_3_assunzioni_prog,      -- raw
   w."q_2_6"::text                                                      AS q2_4_assunzioni_turnover_tot, -- r
   CASE WHEN w."q_2_5" = 1 THEN 'Sì' ELSE 'No' END                     AS q2_5_assessment,           -- raw
   w."q_2_1"::text                                                      AS q2_5_assunzioni_su_prog,   -- raw
   w."q_2_1"::text                                                      AS q2_assunzioni_totali,      -- raw
   w."q_3_4"::text                                                      AS q3_1_concorsi_comp_trasv,  -- raw
   CASE WHEN w."q_3_5" = 1 THEN 'Sì' ELSE 'No' END                     AS q3_2_onboarding,           -- raw
   CASE WHEN w."q_3_6" = 1 THEN 'Sì' ELSE 'No' END                     AS q3_3_apprendistato,        -- raw
   w."q_3_2"::text                                                      AS q3_4_concorsi_profili_cb,  -- raw
   w."q_3_3"::text                                                      AS q3_5_concorsi_dizionario,  -- raw
   w."q_3_1"::text                                                      AS q3_concorsi_totali,        -- raw
   CASE WHEN w."q_4_1" = 1 THEN 'Sì' ELSE 'No' END                     AS q4_1_rilevazione_gap,      -- raw
   CASE WHEN COALESCE(w."q_4_3",0) > 0 THEN 'Sì' ELSE 'No' END         AS q4_2_formazione_trasv,     -- raw
   w."q_4_2"::text                                                      AS q4_percorsi_totali,        -- raw
   CASE WHEN w."q_5_1" = 1 THEN 'Sì' ELSE 'No' END                     AS q5_1_integrazione_performance, --
   CASE WHEN w."q_5_2" = 1 THEN 'Sì' ELSE 'No' END                     AS q5_2_incentivazione_non_mon,   --
   CASE WHEN COALESCE(w."q_5_3",0) > 0 THEN 'Sì' ELSE 'No' END         AS q5_3_convenzioni_universita,   --
   w."q_6_2"::text                                                      AS q6_1_processi_semplificati, -- raw
   NULL::text                                                           AS q6_12_donne_agile_pct,      -- TOD
   (COALESCE(w."q_6_20_a",0)+COALESCE(w."q_6_20_b",0)
    +COALESCE(w."q_6_20_c",0)+COALESCE(w."q_6_20_d",0))::text          AS q6_13_sw_hr_nuovi,          -- raw
   w."q_6_14_a"::text                                                   AS q6_14_progressioni_oriz,    -- raw
   w."q_6_14_b"::text                                                   AS q6_14_progressioni_vert,    -- raw
   w."q_6_8"::text                                                      AS q6_15_eq_ep_under35,        -- raw
   w."q_6_16_a"::text                                                   AS q6_16_donne_agile,          -- raw
   w."q_6_10"::text                                                     AS q6_16_mobilita_out,         -- raw
   w."q_6_16_b"::text                                                   AS q6_16_uomini_agile,         -- raw
   w."q_6_17"::text                                                     AS q6_17_gg_agile_donne,       -- raw
   w."q_6_11"::text                                                     AS q6_17_mobilita_in,          -- raw
   w."q_6_4_a"::text                                                    AS q6_18_donne_dirigenti,      -- raw
   w."q_6_18"::text                                                     AS q6_18_gg_totali,            -- raw
   CASE WHEN (COALESCE(w."q_6_19_a",0)+COALESCE(w."q_6_19_b",0)
             +COALESCE(w."q_6_19_c",0)+COALESCE(w."q_6_19_d",0)) > 0
        THEN 'Sì' ELSE 'No' END                                        AS q6_19_strumenti_ict,        -- raw
   w."q_6_13_a"::text                                                   AS q6_20_entrati_mobilita,     -- raw
   w."q_6_3_a"::text                                                    AS q6_3_dirigente,             -- raw
   w."q_6_3_b"::text                                                    AS q6_3_non_dirigente,         -- raw
```

```
    NULL::text                                                 AS q6_4_posti_vacanti_nondir,  -- TOD
    w."q_6_4_a"::text                                          AS q6_4_ti_dir_donne,          -- raw
    w."q_6_4_b"::text                                          AS q6_4_ti_dir_uomini,         -- raw
    w."q_6_4_c"::text                                          AS q6_4_ti_nondir_donne,       -- raw
    w."q_6_4_d"::text                                          AS q6_4_ti_nondir_uomini,      -- raw
    NULL::text                                                 AS q6_5_posti_vacanti_dir,     -- TOD
    w."q_6_5_a"::text                                          AS q6_5_td_dir_donne,          -- raw
    w."q_6_5_b"::text                                          AS q6_5_td_dir_uomini,         -- raw
    w."q_6_6"::text                                            AS q6_6_under35,               -- raw
    w."q_6_7"::text                                            AS q6_7_eq_ep,                 -- raw
    w."q_6_8"::text                                            AS q6_8_eq_ep_under45,         -- raw
    w."q_6_15"::text                                           AS q6_9_lavoro_flessibile,     -- raw
    w."q_6_11"::text                                           AS q6_comandati_in,            -- raw
    w."q_6_10"::text                                           AS q6_comandati_out,           -- raw
    w."q_6_15"::text                                           AS q6_dip_flessibili,          -- raw
    w."q_6_12_a"::text                                         AS q6_entrati,                 -- raw
    (COALESCE(w."q_6_4_a",0)+COALESCE(w."q_6_4_b",0)
     +COALESCE(w."q_6_4_c",0)+COALESCE(w."q_6_4_d",0))::text   AS q6_organico_medio,          -- som
    w."q_6_3_a"::text                                          AS q6_pianta_organica_dir,     -- raw
    w."q_6_3_b"::text                                          AS q6_pianta_organica_nondir,  -- raw
    w."q_6_1"::text                                            AS q6_processi_totali,         -- raw
    (COALESCE(w."q_6_19_a",0)+COALESCE(w."q_6_19_b",0)
     +COALESCE(w."q_6_19_c",0)+COALESCE(w."q_6_19_d",0))::text AS q6_sw_hr_totali,            -- raw
    (COALESCE(w."q_6_4_a",0)+COALESCE(w."q_6_4_b",0)
     +COALESCE(w."q_6_4_c",0)+COALESCE(w."q_6_4_d",0))::text   AS q6_tep_personale,           -- som
    w."q_6_3_a"::text                                          AS q6_totale_dirigenti,        -- raw
    (COALESCE(w."q_6_4_a",0)+COALESCE(w."q_6_4_c",0))::text    AS q6_totale_donne,            -- TI
    w."q_6_12_b"::text                                         AS q6_usciti                   -- raw
FROM w
LEFT JOIN public.dw_ente de ON de.id_ente = w.id_ente;
CREATE UNIQUE INDEX IF NOT EXISTS dw_kpi_rilevazione_id_idx ON public.dw_kpi_rilevazione (id);
```

`public.dw_verifica_indicatori`

Cosa rappresenta: Anagrafica + organico (indici compositi da definire)

Tabelle sorgenti: `public.dw_ente`, `public.dw_kpi_rilevazione`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_verifica_indicatori CASCADE;
CREATE MATERIALIZED VIEW public.dw_verifica_indicatori AS
SELECT
    de.id_ente                                    AS id_ente,
    de.denominazione                              AS denominazione,
    de.tipo_istituzione                           AS tipologia,
    NULL::text                                    AS profilo,          -- TODO: profilo/cluster ente (metodolog
    COALESCE(k.q6_tep_personale::numeric, de.organico_2023)::numeric AS organico_2023, -- personale TI corren
    NULL::numeric AS cgc,
    NULL::numeric AS cqt,
    NULL::numeric AS iac,
    NULL::numeric AS iap,
    NULL::numeric AS icec,
    NULL::numeric AS icpr,
    NULL::numeric AS icq,
    NULL::numeric AS ics_norm,
    NULL::numeric AS idc,
    NULL::numeric AS idla,
    NULL::numeric AS idp_norm,
    NULL::numeric AS ief_norm,
    NULL::numeric AS iesf,
    NULL::numeric AS ifm_norm,
    NULL::numeric AS igf,
    NULL::numeric AS ipd,
```

```
   NULL::numeric AS irg_norm,
   NULL::numeric AS irs,
   NULL::numeric AS isg,
   NULL::numeric AS pti,
   NULL::numeric AS tcf,
   NULL::numeric AS tcp_gg,
   NULL::numeric AS tcpb,
   NULL::numeric AS tep,
   NULL::numeric AS tsc,
   NULL::numeric AS dpi_norm,
   NULL::text AS cluster_iap,
   NULL::text AS cluster_isg,
   NULL::text AS cluster_tep
FROM public.dw_ente de
LEFT JOIN public.dw_kpi_rilevazione k ON k.id_ente = de.id_ente;
CREATE UNIQUE INDEX IF NOT EXISTS dw_verifica_indicatori_id_ente_idx
   ON public.dw_verifica_indicatori (id_ente);
```

`public.dw_occupazione`

Cosa rappresenta: Personale a tempo pieno / part-time

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_occupazione`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_occupazione CASCADE;
CREATE MATERIALIZED VIEW public.dw_occupazione AS
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()                AS id,
 f.anno                              AS anno,
 f."CONTRATTO"                        AS contratto,
 m.id_ente                           AS istituzione,
 NULL::text                          AS macrocat,           -- non presente in ft_occupazione
 f."PART_TIME_INF50__DONNE"           AS pt_inf50_d,
 f."PART_TIME_INF50__UOMINI"          AS pt_inf50_u,
 f."PART_TIME_SUP50__DONNE"           AS pt_sup50_d,
 f."PART_TIME_SUP50__UOMINI"          AS pt_sup50_u,
 f."QUALIFICA"                        AS qualifica,
 f."PERSONALE_TEMPO_PIENO_DONNE"      AS tp_donne,
 f."PERSONALE_TEMPO_PIENO_UOMINI"     AS tp_uomini
FROM ca.ft_occupazione f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_occupazione_id_idx ON public.dw_occupazione (id);
CREATE INDEX IF NOT EXISTS dw_occupazione_ist_idx ON public.dw_occupazione (istituzione, anno);
```

`public.dw_assunti`

Cosa rappresenta: Assunzioni per anno/causale

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_assunzioni`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_assunti CASCADE;
CREATE MATERIALIZED VIEW public.dw_assunti AS
```

```
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()   AS id,
 f.anno                 AS anno,
 f."CATEGORIA"           AS categoria,
 f."CAUSALE_ASSUNZIONE"  AS causale,
 f."CONTRATTO"           AS contratto,
 f."DONNE"               AS donne,
 m.id_ente              AS istituzione,
 f."QUALIFICA"           AS qualifica,
 f."UOMINI"              AS uomini
FROM ca.ft_assunzioni f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_assunti_id_idx ON public.dw_assunti (id);
CREATE INDEX IF NOT EXISTS dw_assunti_ist_idx ON public.dw_assunti (istituzione, anno);
```

`public.dw_cessati`

Cosa rappresenta: Cessazioni per anno/causale

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_cessazioni`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_cessati CASCADE;
CREATE MATERIALIZED VIEW public.dw_cessati AS
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()   AS id,
 f.anno                 AS anno,
 f."CATEGORIA"           AS categoria,
 f."CAUSALE_CESSAZIONE"  AS causale,
 f."CONTRATTO"           AS contratto,
 f."DONNE"               AS donne,
 m.id_ente              AS istituzione,
 f."QUALIFICA"           AS qualifica,
 f."UOMINI"              AS uomini
FROM ca.ft_cessazioni f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_cessati_id_idx ON public.dw_cessati (id);
CREATE INDEX IF NOT EXISTS dw_cessati_ist_idx ON public.dw_cessati (istituzione, anno);
```

`public.dw_eta`

Cosa rappresenta: Distribuzione per fascia d'eta

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_eta`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_eta CASCADE;
CREATE MATERIALIZED VIEW public.dw_eta AS
```

```
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()   AS id,
 f.anno                 AS anno,
 f."CONTRATTO"           AS contratto,
 f."DONNE"               AS donne,
 f."Fascia_Eta"          AS fascia_eta,
 m.id_ente              AS istituzione,
 f."UOMINI"              AS uomini
FROM ca.ft_eta f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_eta_id_idx ON public.dw_eta (id);
CREATE INDEX IF NOT EXISTS dw_eta_ist_idx ON public.dw_eta (istituzione, anno);
```

`public.dw_formazione`

Cosa rappresenta: Formazione erogata (giornate medie)

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_formazione`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_formazione CASCADE;
CREATE MATERIALIZED VIEW public.dw_formazione AS
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()   AS id,
 f.anno                 AS anno,
 f."CATEGORIA"           AS categoria,
 f."CAUSALE_ASSENZA"     AS causale,
 f."CONTRATTO"           AS contratto,
 f."FORM_DONNE"          AS form_donne,
 f."FORM_UOMINI"         AS form_uomini,
 m.id_ente              AS istituzione,
 f."FORM_MEDIA_DONNE"    AS ore_media_d,
 f."FORM_MEDIA_UOMINI"   AS ore_media_u,
 f."QUALIFICA"           AS qualifica
FROM ca.ft_formazione f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_formazione_id_idx ON public.dw_formazione (id);
CREATE INDEX IF NOT EXISTS dw_formazione_ist_idx ON public.dw_formazione (istituzione, anno);
```

`public.dw_modalita_lavoro`

Cosa rappresenta: Lavoro agile/telelavoro/turnazione/reperibilita

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_modalita_lavoro_flessibile`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_modalita_lavoro CASCADE;
CREATE MATERIALIZED VIEW public.dw_modalita_lavoro AS
```

```sql
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()               AS id,
 f.anno                            AS anno,
 f."CATEGORIA"                      AS categoria,
 f."CONTRATTO"                      AS contratto,
 m.id_ente                         AS istituzione,
 f."PERS_LAVORO_AGILE_D"            AS lavoro_agile_d,
 f."PERS_LAVORO_AGILE_U"            AS lavoro_agile_u,
 f."MACROCATEGORIA"                 AS macrocat,
 f."SOGGETTI_REPERIBILITA_DONNE"    AS reperibilita_d,
 f."SOGGETTI_REPERIBILITA_UOMINI"   AS reperibilita_u,
 f."TELE_LAVORO_DONNE"              AS telelavoro_d,
 f."TELE_LAVORO_UOMINI"             AS telelavoro_u,
 f."SOGGETTI_TURNAZIONE_DONNE"      AS turnazione_d,
 f."SOGGETTI_TURNAZIONE_UOMINI"     AS turnazione_u
FROM ca.ft_modalita_lavoro_flessibile f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_modalita_lavoro_id_idx ON public.dw_modalita_lavoro (id);
CREATE INDEX IF NOT EXISTS dw_modalita_lavoro_ist_idx ON public.dw_modalita_lavoro (istituzione, anno);
```

`public.dw_passaggi_qualifica`

Cosa rappresenta: Progressioni verticali/orizzontali

Tabelle sorgenti: `ca.lk_istituzioni`, `dwh.lk_questionari_enti`, `ca.ft_passaggi_qualifica`

Query SQL:

```sql
DROP MATERIALIZED VIEW IF EXISTS public.dw_passaggi_qualifica CASCADE;
CREATE MATERIALIZED VIEW public.dw_passaggi_qualifica AS
WITH map AS (
 SELECT DISTINCT ON (i."istituzione_id") i."istituzione_id" AS ist_code, qe.id AS id_ente
 FROM ca.lk_istituzioni i
 JOIN dwh.lk_questionari_enti qe ON qe.codice_fiscale_ente = i."CODI_FISCALE"
 ORDER BY i."istituzione_id", qe.id
)
SELECT
 ROW_NUMBER() OVER ()           AS id,
 f.anno                         AS anno,
 f."CATEGORIA_ARRIVO"           AS cat_arrivo,
 f."CATEGORIA_PARTENZA"         AS cat_partenza,
 f."CONTRATTO"                  AS contratto,
 m.id_ente                      AS istituzione,
 f."NUMERO_PASSAGGI"            AS numero_passaggi,
 f."QUALIFICA_ARRIVO"           AS qual_arrivo,
 f."QUALIFICA_PARTENZA"         AS qual_partenza,
 CASE f."TIPO_PASSAGGIO"
      WHEN 'V' THEN 'Verticale'
      WHEN 'O' THEN 'Orizzontale'
      ELSE f."TIPO_PASSAGGIO" END AS tipo_passaggio
FROM ca.ft_passaggi_qualifica f
JOIN map m ON m.ist_code = f."ISTITUZIONE";
CREATE UNIQUE INDEX IF NOT EXISTS dw_passaggi_qualifica_id_idx ON public.dw_passaggi_qualifica (id);
CREATE INDEX IF NOT EXISTS dw_passaggi_qualifica_ist_idx ON public.dw_passaggi_qualifica (istituzione, anno
```

`public.dw_causali`

Cosa rappresenta: Lookup descrizioni causali assunzione/cessazione

Tabelle sorgenti: `ca.vw_lk_causali_assunzione`, `ca.vw_lk_causali_cessazione`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_causali CASCADE;
CREATE MATERIALIZED VIEW public.dw_causali AS
WITH src AS (
 SELECT DISTINCT
   trim(v."causale_id")                        AS cod_alfa,
   trim(both '"' FROM trim(v."DESC_CAUSALE"))  AS descrizione,
   v."TIPO_CAUSALE"                            AS tipo
 FROM ca.vw_lk_causali_assunzione v
 UNION
 SELECT DISTINCT
   trim(v."causale_id"),
   trim(both '"' FROM trim(v."DESC_CAUSALE")),
   v."TIPO_CAUSALE"
 FROM ca.vw_lk_causali_cessazione v
)
SELECT
 ROW_NUMBER() OVER (ORDER BY tipo, cod_alfa) AS id,
 cod_alfa,
 descrizione,
 NULL::int   AS is_ti,   -- flag tempo indeterminato non presente in fonte
 tipo
FROM src;
CREATE UNIQUE INDEX IF NOT EXISTS dw_causali_id_idx ON public.dw_causali (id);
CREATE INDEX IF NOT EXISTS dw_causali_cod_idx ON public.dw_causali (cod_alfa);
```

`public.dw_comparto_contratto`

Cosa rappresenta: Lookup comparto/contratto

Tabelle sorgenti: `ca.lk_mappa_comparti_contratti`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_comparto_contratto CASCADE;
CREATE MATERIALIZED VIEW public.dw_comparto_contratto AS
WITH src AS (
 SELECT DISTINCT
   m."CODI_COMPARTO"    AS cod_comparto,
   m."CODI_CONTRATTO"   AS cod_contratto,
   trim(both '"' FROM m."DESC_COMPARTO")   AS desc_comparto,
   trim(both '"' FROM m."DESC_CONTRATTO")  AS desc_contratto
 FROM ca.lk_mappa_comparti_contratti m
)
SELECT
 ROW_NUMBER() OVER (ORDER BY cod_comparto, cod_contratto) AS id,
 cod_comparto,
 cod_contratto,
 desc_comparto,
 desc_contratto
FROM src;
CREATE UNIQUE INDEX IF NOT EXISTS dw_comparto_contratto_id_idx ON public.dw_comparto_contratto (id);
```

`public.dw_fascia_eta`

Cosa rappresenta: Lookup fasce d'eta (E0..E68)

Tabelle sorgenti: *nessuna (lookup statica generata)*

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_fascia_eta CASCADE;
CREATE MATERIALIZED VIEW public.dw_fascia_eta AS
SELECT * FROM (
 VALUES
   ('E0',  'Fino a 19 anni', 0,   19),
   ('E20', '20-24 anni',     20,  24),
   ('E25', '25-29 anni',     25,  29),
   ('E30', '30-34 anni',     30,  34),
   ('E35', '35-39 anni',     35,  39),
   ('E40', '40-44 anni',     40,  44),
   ('E45', '45-49 anni',     45,  49),
   ('E50', '50-54 anni',     50,  54),
   ('E55', '55-59 anni',     55,  59),
   ('E60', '60-64 anni',     60,  64),
   ('E65', '65-67 anni',     65,  67),
   ('E68', '68 anni e oltre',68,  NULL)
) AS t(codice, classe, eta_min, eta_max);
CREATE UNIQUE INDEX IF NOT EXISTS dw_fascia_eta_codice_idx ON public.dw_fascia_eta (codice);
```

`public.dw_qualifiche`

Cosa rappresenta: Lookup qualifiche -> ruolo per esteso

Tabelle sorgenti: `ca.lk_comparti_categorie_contratti`

Query SQL:

```
DROP MATERIALIZED VIEW IF EXISTS public.dw_qualifiche CASCADE;
CREATE MATERIALIZED VIEW public.dw_qualifiche AS
WITH src AS (
 SELECT DISTINCT
   q."CODI_CONTRATTO"                             AS cod_contratto,
   q."CODI_QUALIFICA"                             AS categoria,      -- codice qualifica (join key)
   trim(both '"' FROM trim(q."DESC_QUALIFICA"))     AS descrizione,    -- ruolo per esteso
   trim(both '"' FROM trim(q."DESC_MACROCATEGORIA")) AS macrocategoria
 FROM ca.lk_comparti_categorie_contratti q
)
SELECT
 ROW_NUMBER() OVER (ORDER BY cod_contratto, categoria) AS id,
 categoria,
 cod_contratto,
 descrizione,
 macrocategoria
FROM src;
CREATE UNIQUE INDEX IF NOT EXISTS dw_qualifiche_id_idx ON public.dw_qualifiche (id);
CREATE INDEX IF NOT EXISTS dw_qualifiche_join_idx ON public.dw_qualifiche (cod_contratto, categoria);
```